Business requirements: sales analytics with SQL Server

Database DataWarehouseAnalytics, schema gold. Scripts 00 to 13, plus customer and product reports.

Item	Detail
Source scripts	scripts/00_init_database.sql through scripts/13_report_products.sql
Tables	gold.dim_customers, gold.dim_products, gold.fact_sales
Views delivered	gold.report_customers, gold.report_products
Engine	SQL Server 2019 or later (T-SQL)
Version	1.0, 24 Sep 2026
Owner	Business analyst. Sign-off: sales lead, product lead, DBA.

Read this before you run the scripts. It states the business questions, the KPI formulas exactly as coded, and what counts as done for each script.

Contents: 1 Business problem. 2 Problem statement and objectives. 3 Scope and stakeholders. 4 Data model. 5 Functional requirements FR-00 to FR-13. 6 KPI definitions. 7 Reporting requirements. 8 Reproducibility, risks, open questions, to-do list.

1. Business problem

Sales, customer, and product records sit in three gold tables with no agreed totals. Each team counts orders, sales, and active customers in its own way, so monthly reviews start with a debate about numbers instead of decisions.

There is also no shared definition of a good customer or a weak product. VIP status, cost bands, and performer tiers change from one spreadsheet to the next. Marketing cannot target, sales cannot prioritize, and product cannot decide what to keep or drop.

This work fixes that by giving one set of SQL queries with fixed formulas. Exploration scripts 01 to 04 confirm what data exists and what the headline totals are. Analysis scripts 05 to 11 explain mix, rank, trend, and share. Views 12 and 13 store the agreed customer and product logic for reuse.

2. Problem statement and objectives

Problem statement: managers cannot answer four questions from the current data without manual work: who buys, what sells, how sales move over time, and which groups drive revenue.

Objectives:

ID	Objective	Done when
O-1	Confirm coverage: tables, columns, date span, customer age span.	FR-01 to FR-03 run and outputs are filed.
O-2	Publish one set of headline totals for sales, orders, quantity, price, products, customers.	FR-04 summary returns 6 rows with stated labels.
O-3	Explain mix and change: distribution, rank, monthly trend, running totals, year-on-year flags, segments, category share.	FR-05 to FR-11 run in order and reconcile.
O-4	Deliver two views for downstream use in dashboards and campaigns.	gold.report_customers and gold.report_products create without error and return one row per key.

3. Scope and stakeholders

In scope

Script 00 rebuild of the database and load of the three gold CSVs. Scripts 01 to 11 for exploration and analysis. Scripts 12 and 13 for the two views. All work in T-SQL on SQL Server.

Out of scope

New source systems, ETL repair, BI dashboard design, forecasts or predictive models, row-level security, and GDPR erasure workflows. Dashboard teams consume the two views but design work is separate.

Stakeholders

Role	Interest	Sign-off
Business analyst	Owns this document and the requirement IDs.	All sections.
Data analyst	Runs scripts 00 to 13 in order.	FR-00 to FR-13 acceptance.
Sales / CRM lead	Uses top customers, VIP list, recency for retention.	VIP thresholds, recency definition.
Product / category lead	Uses top and weak products, YoY flags, performer tiers.	50000 / 10000 bands, cost bands.
IT / DBA	Runs script 00, grants BULK INSERT rights, sets file paths.	Drop-and-recreate step, server paths.

4. Data model

Star schema. Fact grain is one row per order line. Dimensions join with LEFT JOIN from fact to dimension.

Table	Key columns	Date columns
gold.dim_customers	customer_key (join key), customer_id, customer_number, first_name, last_name, country, marital_status, gender	birthdate, create_date
gold.dim_products	product_key (join key), product_id, product_number, product_name, category_id, category, subcategory, maintenance, cost (int), product_line	start_date
gold.fact_sales	order_number, product_key, customer_key, sales_amount (int), quantity (tinyint), price (int)	order_date, shipping_date, due_date

Joins: fact_sales.customer_key = dim_customers.customer_key. fact_sales.product_key = dim_products.product_key. Time logic excludes rows where order_date IS NULL. Those rows are counted separately as a quality check and do not enter trends, running totals, YoY, or the two views.

Count rules used throughout: orders use COUNT(DISTINCT order_number). Ordering customers use COUNT(DISTINCT customer_key) on fact_sales. All customers use COUNT on dim_customers. The distinction matters because some customers in dim_customers have placed no order.

5. Functional requirements

Each requirement maps to one script. Run order is numeric. Each item lists the task and the acceptance check.

ID / script	Task (to do)	Acceptance
FR-00 00_init	Drop and recreate DataWarehouseAnalytics. Create schema gold and 3 tables per DDL. BULK INSERT the 3 gold CSVs with FIRSTROW = 2.	Runs end to end. Row counts match CSVs minus header. Backup step recorded.
FR-01 01_explore	List tables from INFORMATION_SCHEMA.TABLES. List columns for dim_customers from INFORMATION_SCHEMA.COLUMNS.	Output shows catalog, schema, name, type; column name, type, nullability, length.
FR-02 02_dims	SELECT DISTINCT country ordered. SELECT DISTINCT category, subcategory, product_name ordered.	No duplicates. Sort matches spec.
FR-03 03_dates	MIN/MAX order_date with DATEDIFF MONTH span. MIN/MAX birthdate with age via GETDATE().	First/last order and month span filed. Oldest/youngest age shown.
FR-04 04_measures	SUM sales, SUM quantity, AVG price, COUNT and COUNT DISTINCT orders, product and customer counts, UNION ALL 6-row summary.	Summary has 6 labeled rows. Distinct-order logic used.
FR-05 05_magnitude	GROUP BY country, gender, category; AVG cost by category; revenue by category, customer, country.	Each query groups and orders DESC. Join keys stated.
FR-06 06_ranking	TOP 5 products, RANK() variant filtered to rank <= 5, bottom 5 products, TOP 10 customers, bottom 3 by order count.	TOP and RANK top-5 sets match. Tie handling noted.
FR-07 07_trends	Monthly sales, customers, quantity via YEAR/MONTH, DATETRUNC(month), FORMAT yyyy-MMM where order_date IS NOT NULL.	Three variants reconcile to same monthly totals.
FR-08 08_cumulative	Yearly totals with SUM OVER (ORDER BY) running total and AVG OVER moving average price.	Running total equals cumulative sum in date order.
FR-09 09_YoY	CTE yearly_product_sales. AVG OVER (PARTITION BY product) and LAG prior year. Flags Above/Below Avg, Increase/Decrease/No Change.	Diff columns equal current minus avg and current minus prior year. First year py_sales is null.
FR-10 10_segments	Cost bands Below 100 / 100-500 / 500-1000 / Above 1000. Customer VIP / Regular / New from lifespan and spend.	Band counts sum to product total. Segment counts sum to ordering-customer total.
FR-11 11_share	Sales by category with SUM OVER () overall and ROUND(share * 100, 2).	Percents sum to 100 within 0.05 rounding. Ordered DESC.
FR-12 12_cust_view	Drop and CREATE VIEW gold.report_customers. Base join, per-customer aggregation, age groups, segment, recency, AOV, monthly spend.	One row per customer_key. Null dates excluded. Divide-by-zero handled.
FR-13 13_prod_view	Drop and CREATE VIEW gold.report_products. Base join, per-product aggregation, performer tier, recency, AOR, monthly revenue.	One row per product_key. Thresholds 10000 / 50000 applied as coded.

6. KPI definitions

Formulas match the SQL exactly. Grain states the row the number describes.

KPI	Formula in SQL	Grain
Total sales	SUM(sales_amount) on fact_sales	All filtered rows
Total quantity	SUM(quantity)	All filtered rows
Average price	AVG(price)	All filtered rows
Total orders	COUNT(DISTINCT order_number)	All filtered rows
Total products	COUNT(DISTINCT product_name) on dim_products	Product table
Total customers (all)	COUNT(customer_key) on dim_customers	Customer table
Ordering customers	COUNT(DISTINCT customer_key) on fact_sales	Fact table
Lifespan (customer)	DATEDIFF(month, MIN(order_date), MAX(order_date)) per customer	Customer
Recency (customer)	DATEDIFF(month, last_order_date, GETDATE())	Customer
Avg order value	total_sales / total_orders; 0 when total_orders = 0	Customer
Avg monthly spend	total_sales / lifespan; total_sales when lifespan = 0	Customer
Lifespan (product)	DATEDIFF(month, MIN(order_date), MAX(order_date)) per product	Product
Recency (product)	DATEDIFF(month, last_sale_date, GETDATE())	Product
Avg order revenue	total_sales / total_orders; 0 when total_orders = 0	Product
Avg monthly revenue	total_sales / lifespan; total_sales when lifespan = 0	Product
Avg selling price	ROUND(AVG(sales_amount / NULLIF(quantity, 0)), 1)	Product
Category share	ROUND(total_sales / SUM(total_sales) OVER () * 100, 2)	Category

7. Reporting requirements

Customer view: gold.report_customers (script 12)

Base: fact_sales LEFT JOIN dim_customers on customer_key, where order_date IS NOT NULL. Fields: customer_key, customer_number, customer_name (first + last), age = DATEDIFF(year, birthdate, GETDATE()). Aggregation per customer: total_orders (distinct order_number), total_sales, total_quantity, total_products (distinct product_key), lifespan, last_order_date. Final select adds age_group, customer_segment, recency, avg_order_value, avg_monthly_spend.

Age groups: Under 20 (age < 20), 20-29, 30-39, 40-49, 50 and above. Segments: VIP (lifespan >= 12 and total_sales > 5000), Regular (lifespan >= 12 and total_sales <= 5000), New (else, lifespan < 12). Sales and marketing use VIP for loyalty tiers, Regular for upsell, New with high recency for onboarding checks.

Product view: gold.report_products (script 13)

Base: fact_sales LEFT JOIN dim_products on product_key, where order_date IS NOT NULL. Fields: product_key, product_name, category, subcategory, cost. Aggregation per product: lifespan, last_sale_date, total_orders, total_customers (distinct customer_key), total_sales, total_quantity, avg_selling_price. Final select adds recency_in_months, product_segment, avg_order_revenue, avg_monthly_revenue.

Tiers, checked in this order: High-Performer (total_sales > 50000), then Mid-Range (total_sales >= 10000, so 10000 to 50000 once High rows are taken out), else Low-Performer. Product leads use High-Performer for stock priority, Low-Performer with high recency for delist review. Thresholds 10000 and 50000 are business parameters and can change after sign-off.

8. Reproducibility, risks, open questions, to-do list

Reproducibility

Scripts carry headers with purpose, tables, and functions. Views use IF OBJECT_ID(..., 'V') DROP VIEW before CREATE VIEW, so reruns are safe. Reruns on the same CSVs return the same results except recency and age, which use GETDATE() and shift daily. SQL Server 2019 or later is required for DATETRUNC and FORMAT.

Risks

Risk	Effect	Mitigation
Script 00 drops the database.	Data loss if run on the wrong server.	DBA confirms server name. Back up first.
Hard-coded C:\sql\... paths in BULK INSERT.	Load fails on other machines.	Edit the 3 paths before run. Use a server-local folder.
GETDATE() in recency and age.	Values drift daily; reports do not match.	File the run date with each export.
Missing comma in script 12, final select.	View fails to create.	Add comma after total_products, retest. See to-do T-1.
int / tinyint limits, FORMAT sorting.	Large sales overflow; yyyy-MMM sorts as text.	Check max sales_amount. Sort trends by year/month columns.

Open questions

#	Question	Owner
Q-1	Confirm VIP rule: lifespan 12+ months and spend over 5000. Currency Euro?	Sales lead
Q-2	Confirm cost bands (100 / 500 / 1000) and product tiers (10000 / 50000).	Product lead
Q-3	Confirm age bands. Keep 5 groups as coded?	Sales lead
Q-4	Include null-date orders anywhere, or keep excluded from time logic?	Data analyst
Q-5	Round category share to 2 decimals. Accept 0.05 total drift?	Finance

To-do list

ID	Task	Depends on
T-1	Fix missing comma in 12_report_customers.sql and recreate gold.report_customers.	FR-12
T-2	Set BULK INSERT paths for your server and run 00. Record row counts for the 3 tables.	FR-00
T-3	Run 01 to 04. File first/last order date, month span, and the 6-row headline summary.	O-1, O-2
T-4	Run 05 to 11. File top/bottom lists, monthly trend, running totals, YoY flags, segments, shares.	O-3
T-5	Run 12 and 13. Export both views. Check one-row-per-key and divide-by-zero cases.	O-4
T-6	Close Q-1 to Q-5 with owners. Bump this document to 1.1 after sign-off.	All above

Appendix: table DDL is in scripts/00_init_database.sql lines 38 to 78. CSV names: gold.dim_customers.csv, gold.dim_products.csv, gold.fact_sales.csv in datasets/csv-files/. Detail column lists are in section 4.